

FORMATION PYTHON

J4

Objets et analyse de données

Modéliser le métier avec des dataclasses, puis nettoyer, agréger et joindre les fichiers avec pandas. Lancement du projet final.

08-poo

09-pandas-analyse

projet-final

Objectifs de la journée



Modéliser avec des classes

Contribuable, Declaration : données + comportements ensemble.



Nettoyer avec pandas

Doublons, valeurs manquantes, libellés incohérents — en 3 lignes.



Joindre les fichiers

déclarations × contribuables × régions : le merge.



Lancer le projet

Tirage des sujets en fin de matinée — 5 sujets, 2-3 par groupe.

LA SEMAINE

J1

Fondamentaux du langage

J2

Fonctions & structures

J3

Fichiers & robustesse

J4

Objets & pandas

J5

Intégration & projets

Après-midi : premier jalon du projet.

@dataclass : la fiche qui se défend



__init__ écrit pour vous

Les champs annotés suffisent : `Declaration("2025-01", ...)`.



@property : valeur calculée

`d.tva_nette` se LIT comme un champ, se CALCULE comme une méthode.



Jamais de liste en défaut

`field(default_factory=list)` — le piège classique de Python.



Quand ne PAS faire de classe

Un dict suffit si aucune règle ne protège les données.



tp_contribuable.py

```
@dataclass
class Declaration:
    periode: str
    chiffre_affaires: int
    tva_collectee: int
    tva_deductible: int

    @property
    def tva_nette(self) -> int:
        return max(self.tva_collectee
                    - self.tva_deductible, 0)
```

pandas : charger et inspecter



Le DataFrame : un tableau typé

Colonnes nommées, opérations vectorisées — fini la boucle ligne à ligne.



dtype={"nif": str} au chargement

Sinon les NIF deviennent des nombres et perdent leurs zéros.



info() et describe() d'abord

Types, valeurs manquantes, ordres de grandeur — AVANT de calculer.



Filtrer par masque

df[df["chiffre_affaires"] > 0] : la condition indexe le tableau.



tp_analyse.py

```
import pandas as pd

decl = pd.read_csv(
    DATA / "declarations.csv",
    dtype={"nif": str})

decl.info()      # 60 000 lignes
decl["tva_collectee"].isna().sum()
1802             # manquantes !

valides = decl[decl["chiffre_affaires"] > 0]
```

Nettoyer : tout le J3, en trois lignes



drop_duplicates(subset="nif")

Les doubles saisies du fichier contribuables disparaissent.



.str.strip().str.upper()

« commerce gal », « COMMERCE GENERAL » → une seule catégorie.



fillna(0)

Les TVA collectées vides deviennent exploitables.



.clip(lower=0)

La TVA nette ne descend jamais sous zéro — comme au module 08.



nettoyage

```
contr = contr.drop_duplicates(  
    subset="nif")  
  
contr["secteur"] = (contr["secteur"]  
    .str.strip().str.upper())  
  
decl["tva_collectee"] = \  
    decl["tva_collectee"].fillna(0)  
  
decl["tva_nette"] = (decl["tva_collectee"]  
    - decl["tva_deductible"]).clip(lower=0)
```

Agréger et joindre : le rapport TVA



groupby = le comptage du J2

compte.get(k, 0) + 1 devient `.groupby("region").sum()`.



merge = la jointure

déclarations ← contribuables (sur nif) ← régions (sur code).



how="left" révèle les orphelins

raison_sociale manquante = déclaration sans contribuable.



Export en une ligne

`to_csv`, `to_excel` : le rapport part chez le destinataire.



le rapport

```
ens = decl.merge(contr, on="nif",  
                 how="left")  
ens = ens.merge(regions, on="code_region")
```

```
rapport = (ens  
           .groupby("region")["tva_nette"]  
           .sum()  
           .sort_values(ascending=False))
```

```
Gabú          6.84e+09  
Oio           6.82e+09  ...
```

Cinq sujets, tirés au sort

1 • Défaillants

Qui n'a rien déposé sur la période ? Par région, par secteur.

2 • Crédits de TVA

Où la déductible dépasse-t-elle durablement la collectée ?

3 • Recouvrement

Déclarations × paiements : taux de couverture, restes à payer.

4 • Qualité des données

Mesurer et corriger : doublons, dates, libellés, orphelins.

5 • Tableau de bord régional

CA, TVA, top secteurs, évolution — export Excel multi-feuilles.

Les jalons — et le premier est aujourd'hui



J4 midi

Sujet tiré, dépôt du groupe créé,
socle exécuté tel quel



J4 soir

Lecture + validation
opérationnelles sur les vraies
données



J5 midi

Chaîne complète exécutable en
UNE commande



J5 après-midi

Soutenance : 10 min de
démonstration + 5 min de
questions

Règle absolue : les lignes écartées sont comptées et affichées — jamais perdues en silence. Le programme échoue proprement si les fichiers manquent.

Objets, pandas — et le projet démarre

1 **tp_contribuable.py** — tva_nette, ca_total, tva_due_totale — assert fournis

2 **tp_analyse.py** — le rapport TVA par région, de bout en bout

3 **Tirage des sujets** — fin de matinée — constituez les groupes AVANT

4 **Jalon du soir** — socle exécuté + lecture/validation sur vraies données



Fichiers du jour

```
08-poo/  
    tp_contribuable.py  
09-pandas-analyse/  
    tp_analyse.py  
projet-final/consignes.md
```



Comparez vos comptages pandas avec ceux du J3 « à la main » : mêmes chiffres = confiance.

À retenir

- ✓ **dataclass + property** — la TVA nette se protège toute seule : jamais négative, partout.
- ✓ **dtype={"nif": str}** — LE piège pandas des identifiants — vu une fois, plus jamais oublié.
- ✓ **Nettoyage en 3 lignes** — `drop_duplicates`, `str.upper`, `fillna` : tout le J3, vectorisé.
- ✓ **merge + groupby** — trois fichiers joints, un rapport par région trié.
- ✓ **Le projet est lancé** — sujet tiré, socle en main, premier jalon franchi ce soir.



Demain — J5 : intégration et soutenances

La chaîne complète en une commande (`argparse`, échec propre), la matinée pour finir, les soutenances l'après-midi.